Survey on the Quality of Automatically Generated DSLs

Q1. Survey Statement

Thank you for participating in this survey! The purpose of this questionnaire is to evaluate the output quality of our DSL generation tool. Your feedback will directly help us improve the tool's usability, accuracy, and functionality.
About your data:
1.Completely Anonymous: This survey does not collect any personally identifiable information (e.g., name, email). All responses are processed anonymously.
2.Use of Data: Your responses may be anonymized and used for academic research (e.g., publications) or product improvement analysis, but no personal privacy will be involved.
3.Privacy Protection: The data will only be accessible to our internal team and collaborating researchers. It will not be shared or made public.
Estimated completion time: 5–10 minutes.
If you have any questions, please contact: xxx
Your participation is invaluable to us—thank you for your support! [Single Choice Question]

Options	Subtotal	Proportion
I agree to participate in this survey and allow my responses to be used for research purposes.	15	100%
Disagree	0	0%
Valid response count for this question	15	

Q2. How many years have you worked in the tech industry? [Single Choice Question]

Options	Subtotal	Proportion
0-2 years	3	20%
3-5 years	7	46.67%
6-10 years	3	20%
More than 10 years	2	13.33%
Valid response count for this question	15	

Q3. What is your role/identity? [Single Choice Question]

Options	Subtotal	Proportion
DSL engine developer	5	33.33%
DSL rule developer	6	40%
General developer (DSL rule user)	4	26.67%
Other	0	0%
Valid response count for this question	15	

Q4. How frequently do you use the xxx DSL? [Single Choice Question]

Options	Subtotal	Proportion
Never	0	0%
Daily	6	40%
Weekly	3	20%
Monthly	6	40%
Valid response count for this question	15	

Evaluation of Automatically Generated DSLs

This section contains **10 sets of automatically generated DSL examples.**
Each set includes:
1. A natural language description of a code defect
2. An example fix for the defect (in diff format)
3. Two DSLs generated by different methods (**DSL_A and DSL_B**)

You will evaluate each DSL based on two criteria:
1. Readability: Is the generated DSL clear and easy to understand? Could you use it directly or with minor modifications?
2. Generalizability: Based on your knowledge, can the generated DSL capture similar defects in other code contexts?

Please review each set carefully and answer the following questions.
Your feedback will directly help us improve the quality of DSL generation.

Q5. Example [1/10]

Defect Description:

The code should never throw a NullPointerException under normal circumstances. Catching such exceptions may mask the original issue and lead to more subtle, hard-to-detect problems later on.

Code Defect – Fix Example (- indicates a line removed from the buggy code, + indicates a line added in the fix):

```java
public void testGetUnchecked_null() {
   for (LoadingCache<Object, Object> cache : caches()) {
-      try {
-          cache.getUnchecked(null);
-          fail("Expected NullPointerException");
-      } catch (NullPointerException expected) {
-      }
+     assertThrows(NullPointerException.class, () -> cache.getUnchecked(null));
      checkEmpty(cache);
   }
}
```

To make it easier for you to read, we have annotated the syntactic correctness of the generated DSLs:

Automatically generated **DSL_A** (Syntax Correct):

```dsl
functionDeclaration functiondeclaration_1 where
   functiondeclaration_1.body contain exceptionBlock exceptionblock_1 where
      exceptionblock_1.catchBlocks contain catchBlock catchblock_1 where
         catchblock_1.parameters contain alias_1 where
            and(
               alias_1 is valueDeclaration,
               alias_1.type.name match ".*NullPointerException"
            );
```

Automatically generated **DSL_B** (Syntax Error):

```dsl
tryCatchStatement tc where
   tc.catchClauses contain catchClause cc where
      cc.exceptionType.name == "NullPointerException";
```

Each DSL should be evaluated along two dimensions:
1. Readability: Is the generated DSL clear and easy to understand? Can you use it directly or with minimal modifications?
2. Generalizability: Based on your expertise, can the generated DSL be used to detect similar defects in different code contexts? [Matrix Scale Question]

Question\Options	Very Poor	Poor	Fair	Good	Excellent	Average Score
Readability of DSL_A	0(0%)	0(0%)	9(60%)	5(33.33%)	1(6.67%)	3.47
Generalizability of DSL_A	0(0%)	0(0%)	1(6.67%)	11(73.33%)	3(20%)	4.13
Readability of DSL_B	0(0%)	1(6.67%)	1(6.67%)	6(40%)	7(46.67%)	4.27
Generalizability of DSL_B	0(0%)	2(13.33%)	5(33.33%)	5(33.33%)	3(20%)	3.6
Subtotal	0(0%)	3(5%)	16(26.67%)	27(45%)	14(23.33%)	3.87

Q6. Example [2/10]

Defect Description:
Please avoid using printStackTrace(). Instead, use a logger (e.g., logger.error(...)) to record exceptions.
Code Defect – Fix Example (- indicates a line removed from the buggy code, + indicates a line added in the fix):

```java
public List<Long> getDatabaseNames() {
   ...
      try {
         Thread.sleep(1000);
      } catch (InterruptedException e1) {
-           e1.printStackTrace();
+           LOG.warn("", e1);
      }
   } catch (DatabaseException e) {
      LOG.warn("catch an exception when calling getDatabaseNames", e);
   ...
```

To make it easier for you to read, we have annotated the syntactic correctness of the generated DSLs:

Automatically generated **DSL_A** (Syntax Correct):

```dsl
functionDeclaration functiondeclaration_1 where
   functiondeclaration_1.body contain functionCall functioncall_1 where
      functioncall_1.name == "printStackTrace";
```

Automatically generated **DSL_B** (Syntax Error):

```dsl
functionCall fc where
    and(
        fc.name == "printStackTrace",
```

```
        fc.base is VariableAccess
    );
```

Each DSL should be evaluated along two dimensions:
1. Readability: Is the generated DSL clear and easy to understand? Can you use it directly or with minimal modifications?
2. Generalizability: Based on your expertise, can the generated DSL be used to detect similar defects in different code contexts? [Matrix Scale Question]

Question\Options	Very Poor	Poor	Fair	Good	Excellent	Average Score
Readability of DSL_A	0(0%)	0(0%)	4(26.67%)	6(40%)	5(33.33%)	4.07
Generalizability of DSL_A	0(0%)	0(0%)	1(6.67%)	10(66.67%)	4(26.67%)	4.2
Readability of DSL_B	0(0%)	0(0%)	2(13.33%)	6(40%)	7(46.67%)	4.33
Generalizability of DSL_B	0(0%)	1(6.67%)	3(20%)	5(33.33%)	6(40%)	4.07
Subtotal	0(0%)	1(1.67%)	10(16.67%)	27(45%)	22(36.67%)	4.17

Q7. Example [3/10]
Defect Description:
Use .equals() to compare object values instead of ==, which checks for reference equality.
Code Defect – Fix Example (- indicates a line removed from the buggy code, + indicates a line added in the fix):

```java
private Map<Integer, ReaderInfo> filterRegisteredReaders(Map<Integer, ReaderInfo> readers) {
   Map<Integer, ReaderInfo> readersForSource = new HashMap<>(readers.size());
   for (Map.Entry<Integer, ReaderInfo> e : readers.entrySet()) {
-        if (readerSourceIndex.get(e.getKey()) == (Integer) sourceIndex) {
+        if (Objects.equals(readerSourceIndex.get(e.getKey()), sourceIndex)) {
          readersForSource.put(e.getKey(), e.getValue());
      }
   }
...
}
```

To make it easier for you to read, we have annotated the syntactic correctness of the generated DSLs:

Automatically generated **DSL_A** (Syntax Correct):

```dsl
functionDeclaration functiondeclaration_1 where
   functiondeclaration_1.body contain binaryOperation binaryoperation_1 where
      binaryoperation_1.operator == "==";
```

```
```

Automatically generated DSL_B (Syntax Error):

```dsl
binaryExpression be where
  and(
    be.operator == "==",
    be.leftOperand is functionCall where
    and(
      name == "get",
      base is variableAccess where
        name == "readerSourceIndex"
    ),
    be.rightOperand is castExpression where
    and(
      type.name == "Integer",
      expression is variableAccess where
      name == "sourceIndex"
    )
  );
```

Each DSL should be evaluated along two dimensions:
1. Readability: Is the generated DSL clear and easy to understand? Can you use it directly or with minimal modifications?
2. Generalizability: Based on your expertise, can the generated DSL be used to detect similar defects in different code contexts? [Matrix Scale Question]

Question\Options	Very Poor	Poor	Fair	Good	Excellent	Average Score
Readability of DSL_A	0(0%)	2(13.33%)	1(6.67%)	9(60%)	3(20%)	3.87
Generalizability of DSL_A	1(6.67%)	3(20%)	4(26.67%)	5(33.33%)	2(13.33%)	3.27
Readability of DSL_B	1(6.67%)	5(33.33%)	6(40%)	3(20%)	0(0%)	2.73
Generalizability of DSL_B	5(33.33%)	4(26.67%)	2(13.33%)	4(26.67%)	0(0%)	2.33
Subtotal	7(11.67%)	14(23.33%)	13(21.67%)	21(35%)	5(8.33%)	3.05

Q8. Example [4/10]
Defect Description:
When using logging, always check if the log level is enabled before performing expensive operations like string concatenation or method calls, to avoid unnecessary performance overhead.
Code Defect – Fix Example (- indicates a line removed from the buggy code, + indicates a line added in the fix):

```java
private void addScanRangeLocations(Partition partition,
        List<Tablet> tablets) throws UserException {
```

```
...
            Collections.shuffle(replicas);
        }
    } else {
-       LOG.debug("use fix replica, value: {}, replica count: {}", useFixReplica,
replicas.size());
+       if (LOG.isDebugEnabled()) {
+           LOG.debug("use fix replica, value: {}, replica count: {}", useFixReplica,
replicas.size());
+       }
        // sort by replica id
        replicas.sort(Replica.ID_COMPARATOR);
        Replica replica = replicas.get(useFixReplica >= replicas.size() ? replicas.size() -
1 :                                        useFixReplica);
...
}
```

To make it easier for you to read, we have annotated the syntactic correctness of the generated DSLs:

Automatically generated DSL_A (Syntax Correct):

```dsl
functionDeclaration functiondeclaration_1 where
    functiondeclaration_1.body contain functionCall functioncall_1 where
        and(
            functioncall_1.base.name match "(?i).*(log|logger)$",
            functioncall_1.name match "(?i).*(debug|info|warn|error|trace)",
            functioncall_1 notIn ifBlock ifblock_1 where
            ifblock_1.condition.name == "isDebugEnabled"
        );
```

Automatically generated DSL_B (Syntax Error):

```dsl
functionCall fc where
    and(
        fc.base.name match "log.*",
        fc.enclosingFunction ef where
        not(
            ef contain ifStatement ifs where
            and(
                ifs.condition is functionCall condFc where
                    and(
                        condFc.base.name match "log.*",
                        condFc.name == "isDebugEnabled"
                    ),
                ifs.body contain fc
        )
    )
);
```

Each DSL should be evaluated along two dimensions:
1. Readability: Is the generated DSL clear and easy to understand? Can you use it directly or with minimal modifications?
2. Generalizability: Based on your expertise, can the generated DSL be used to detect similar defects in different code contexts? [Matrix Scale Question]

Question\Options	Very Poor	Poor	Fair	Good	Excellent	Average Score
Readability of DSL_A	0(0%)	0(0%)	1(6.67%)	10(66.67%)	4(26.67%)	4.2
Generalizability of DSL_A	0(0%)	1(6.67%)	1(6.67%)	8(53.33%)	5(33.33%)	4.13
Readability of DSL_B	1(6.67%)	6(40%)	4(26.67%)	4(26.67%)	0(0%)	2.73
Generalizability of DSL_B	1(6.67%)	5(33.33%)	2(13.33%)	5(33.33%)	2(13.33%)	3.13
Subtotal	2(3.33%)	12(20%)	8(13.33%)	27(45%)	11(18.33%)	3.55

Q9. Example [5/10]
Defect Description:
Avoid using .equals("") to check for empty strings. It's less efficient than using .isEmpty().
Code Defect – Fix Example (- indicates a line removed from the buggy code, + indicates a line added in the fix):

```java
private void backupData() throws BackupException {
...
   boolean isExternal = fileName.startsWith(EXTERNAL_PREFIX);
   // Get new file name
   fileName = fileName.replaceFirst((isExternal ? EXTERNAL_PREFIX :
INTERNAL_PREFIX) +
                    Pattern.quote(mPackageName + "/") + "\\./", "");
-  if (fileName.equals("")) continue;
+  if (fileName.isEmpty()) continue;
   // New tar entry
   TarArchiveEntry outTarEntry = new TarArchiveEntry(fileName);
   outTarEntry.setMode(inTarEntry.getMode());
...
}
```

To make it easier for you to read, we have annotated the syntactic correctness of the generated DSLs:

Automatically generated DSL_A (Syntax Correct):

```dsl
functionDeclaration functiondeclaration_1 where
   functiondeclaration_1.body contain ifBlock ifblock_1 where
      and(
         ifblock_1.condition.arguments contain alias_1 where
```

```
            and(
                alias_1 is literal,
                alias_1.value == ""
            ),
        ifblock_1.condition.name == "equals"
    );
```

Automatically generated DSL_B (Syntax Error):

```dsl
ifStatement ifs where
  ifs.condition contain binaryExpression be where
    and(
      be.operator == "==",
      be.leftOperand is stringLiteral where value == "",
      be.rightOperand is variableAccess where name == "fileName"
    );
```

Each DSL should be evaluated along two dimensions:
1. Readability: Is the generated DSL clear and easy to understand? Can you use it directly or with minimal modifications?
2. Generalizability: Based on your expertise, can the generated DSL be used to detect similar defects in different code contexts? [Matrix Scale Question]

Question\Options	Very Poor	Poor	Fair	Good	Excellent	Average Score
Readability of DSL_A	0(0%)	1(6.67%)	2(13.33%)	11(73.33%)	1(6.67%)	3.8
Generalizability of DSL_A	0(0%)	0(0%)	1(6.67%)	13(86.67%)	1(6.67%)	4
Readability of DSL_B	2(13.33%)	2(13.33%)	3(20%)	6(40%)	2(13.33%)	3.27
Generalizability of DSL_B	5(33.33%)	4(26.67%)	2(13.33%)	3(20%)	1(6.67%)	2.4
Subtotal	7(11.67%)	7(11.67%)	8(13.33%)	33(55%)	5(8.33%)	3.37

Q10. Example [6/10]
Defect Description:
Avoid using constructors for boxed primitives; use static factory methods like Integer.valueOf() instead.
Code Defect – Fix Example (- indicates a line removed from the buggy code, + indicates a line added in the fix):

```java
public void allocMeasuring() {
...
  Measurement measurement = createMeasurement();
  measurement.start();
```

```
    for (int i=0; i<numAlloc; i++)
-  new Integer(i);
+  Integer.valueOf(i);
    measurement.stop();
    logger.info(" ** {}", measurement.prettyBytes());
...
}
```

To make it easier for you to read, we have annotated the syntactic correctness of the generated DSLs:

Automatically generated DSL_A (Syntax Correct):

```dsl
functionDeclaration functiondeclaration_1 where
    functiondeclaration_1.body contain
        objectCreationExpression objectcreationexpression_1 where
            objectcreationexpression_1.type.name match
                ".*(Integer|Double|Boolean|Long|Short|Byte|Character)";
```

Automatically generated DSL_B (Syntax Error):

```dsl
objectCreationExpression oce where
    and(
        oce.name in ["Integer", "Long", "Double", "Float", "Short", "Byte",
"Character", "Boolean"],
        not(
            oce in methodCall where
            methodCall.name == "valueOf"
        )
    );
```

Each DSL should be evaluated along two dimensions:
1. Readability: Is the generated DSL clear and easy to understand? Can you use it directly or with minimal modifications?
2. Generalizability: Based on your expertise, can the generated DSL be used to detect similar defects in different code contexts? [Matrix Scale Question]

Question\Options	Very Poor	Poor	Fair	Good	Excellent	Average Score
Readability of DSL_A	0(0%)	0(0%)	3(20%)	10(66.67%)	2(13.33%)	3.93
Generalizability of DSL_A	0(0%)	0(0%)	1(6.67%)	12(80%)	2(13.33%)	4.07

Readability of DSL_B	1(6.67%)	2(13.33%)	3(20%)	6(40%)	3(20%)	3.53
Generalizability of DSL_B	1(6.67%)	4(26.67%)	2(13.33%)	6(40%)	2(13.33%)	3.27
Subtotal	2(3.33%)	6(10%)	9(15%)	34(56.67%)	9(15%)	3.7

Q11. Example [7/10]
Defect Description:
The isEmpty() method provided by java.util.Collection is specifically designed to check whether a collection contains any elements. While using size() == 0 achieves the same functionality, it is less expressive than isEmpty() in conveying the programmer's intent.

Code Defect – Fix Example (- indicates a line removed from the buggy code, + indicates a line added in the fix):

```java
public Collection<Part> getParts() {
   ...
      cachedMultiparts = PartParser.parseFrom(this);
   }

-  return (cachedMultiparts.size() > 0) ? cachedMultiparts : null;
+  return (cachedMultiparts.isEmpty()) ? null : cachedMultiparts;
}
```

To make it easier for you to read, we have annotated the syntactic correctness of the generated DSLs:

Automatically generated DSL_A (Syntax Correct):

```dsl
functionDeclaration functiondeclaration_1 where
   functiondeclaration_1.body contain binaryOperation binaryoperation_1 where
      and(
         binaryoperation_1.lhs.name == "size",
         binaryoperation_1.rhs.value == 0,
         binaryoperation_1.operator == ">"
      );
```

Automatically generated DSL_B (Syntax Error):

```dsl
methodDeclaration md where
   md.contain returnStatement rs where
      rs.contain binaryExpression be where
         and(
            be.operator == ">",
            be.left is functionCall fc where
            and(
               fc.name == "size",
```

```
            fc.base is variableAccess va where
            va.name == "cachedMultiparts"
        ),
        be.right is literal lt where
            lt.value == "0"
    )
);
```

Each DSL should be evaluated along two dimensions:
1. Readability: Is the generated DSL clear and easy to understand? Can you use it directly or with minimal modifications?
2. Generalizability: Based on your expertise, can the generated DSL be used to detect similar defects in different code contexts? [Matrix Scale Question]

Question\Options	Very Poor	Poor	Fair	Good	Excellent	Average Score
Readability of DSL_A	0(0%)	0(0%)	2(13.33%)	5(33.33%)	8(53.33%)	4.4
Generalizability of DSL_A	0(0%)	1(6.67%)	2(13.33%)	6(40%)	6(40%)	4.13
Readability of DSL_B	1(6.67%)	4(26.67%)	7(46.67%)	2(13.33%)	1(6.67%)	2.87
Generalizability of DSL_B	3(20%)	5(33.33%)	3(20%)	3(20%)	1(6.67%)	2.6
Subtotal	4(6.67%)	10(16.67%)	14(23.33%)	16(26.67%)	16(26.67%)	3.5

Q12. Example [8/10]
Defect Description:
The StandardCharsets class provides constants for commonly used character sets such as UTF-8. Using these constants is not only less error-prone, but also slightly more performant than using Charset.forName(...), as it avoids scanning the internal charset cache.
Code Defect – Fix Example (- indicates a line removed from the buggy code, + indicates a line added in the fix):
```java
public static void readAcontainerLogs(DataInputStream valueStream, Writer writer, long logUploadedTime) throws IOException {
...
    OutputStream os = null;
    PrintStream ps = null;
    try {
-  os = new WriterOutputStream(writer, Charset.forName("UTF-8"));
+  os = WriterOutputStream.builder()
+          .setWriter(writer)
+          .setCharset(StandardCharsets.UTF_8)
+          .get();
    ps = new PrintStream(os);
    while (true) {
```

...
}
```

To make it easier for you to read, we have annotated the syntactic correctness of the generated DSLs:

Automatically generated DSL_A (Syntax Correct ):

```dsl
functionDeclaration functiondeclaration_1 where
  functiondeclaration_1.body contain functionCall functioncall_1 where
    and(
      functioncall_1.arguments contain alias_1 where
      and(
        alias_1 is literal,
        alias_1.value match "(?i).*(utf-8|iso-8859-1|us-ascii)"
      ),
      functioncall_1.base.name == "Charset",
      functioncall_1.name == "forName"
    );
```

Automatically generated DSL_B (Syntax Error):

```dsl
functionCall fc where
  and(
    fc.name == "forName",
    fc.base.name == "Charset",
    fc.arguments[0] arg where
    arg == "UTF-8"
  );
```

Each DSL should be evaluated along two dimensions:
1. Readability: Is the generated DSL clear and easy to understand? Can you use it directly or with minimal modifications?
2. Generalizability: Based on your expertise, can the generated DSL be used to detect similar defects in different code contexts? [Matrix Scale Question]

Question\Options	Very Poor	Poor	Fair	Good	Excellent	Average Score
Readability of DSL_A	0(0%)	0(0%)	7(46.67%)	7(46.67%)	1(6.67%)	3.6
Generalizability of DSL_A	0(0%)	0(0%)	3(20%)	10(66.67%)	2(13.33%)	3.93
Readability of DSL_B	1(6.67%)	1(6.67%)	3(20%)	3(20%)	7(46.67%)	3.93
Generalizability of DSL_B	1(6.67%)	5(33.33%)	2(13.33%)	4(26.67%)	3(20%)	3.2
Subtotal	2(3.33%)	6(10%)	15(25%)	24(40%)	13(21.67%)	3.67
```

Q13. Example [9/10]
Defect Description:
This may be a forgotten JUnit test case. The method should be annotated with either @Test to enable execution or @Ignore to explicitly skip it.
Code Defect – Fix Example (- indicates a line removed from the buggy code, + indicates a line added in the fix):

```java
+ @Test
public void testArityException() {
   assertNotNull(ArityException.create(0, 0, -1));
   assertNotNull(ArityException.create(0, 1, -1));
   ...
}
```

To make it easier for you to read, we have annotated the syntactic correctness of the generated DSLs:

Automatically generated DSL_A (Syntax Correct):

```dsl
functionDeclaration functiondeclaration_1 where
   and(
      functiondeclaration_1.name match "(?i).*test.*",
      not(
         functiondeclaration_1 contain annotation annotation_1 where
         annotation_1.name == "Test"
      )
);
```

Automatically generated DSL_B (Syntax Error):

```dsl
functionDeclaration fd where
   and(
      fd.name startWith "test",
      not(
         fd.annotations contain annotation where
         annotation.name == "Test"
      )
   );
```

Each DSL should be evaluated along two dimensions:
1. Readability: Is the generated DSL clear and easy to understand? Can you use it directly or with minimal modifications?
2. Generalizability: Based on your expertise, can the generated DSL be used to detect similar defects in different code contexts? [Matrix Scale Question]

Question\Options	Very Poor	Poor	Fair	Good	Excellent	Average Score
Readability of DSL_A	0(0%)	0(0%)	2(13.33%)	8(53.33%)	5(33.33%)	4.2

Generalizability of DSL_A	0(0%)	0(0%)	1(6.67%)	8(53.33%)	6(40%)	4.33
Readability of DSL_B	1(6.67%)	0(0%)	3(20%)	6(40%)	5(33.33%)	3.93
Generalizability of DSL_B	1(6.67%)	1(6.67%)	2(13.33%)	7(46.67%)	4(26.67%)	3.8
Subtotal	2(3.33%)	1(1.67%)	8(13.33%)	29(48.33%)	20(33.33%)	4.07

Q14. Example [10/10]

Defect Description:

When calling methods that convert strings to numbers, such as Integer.parseInt(), failing to handle NumberFormatException may lead to unexpected runtime errors.

Code Defect – Fix Example (- indicates a line removed from the buggy code, + indicates a line added in the fix):

```java
private int getChunkSize(final Properties props) {
  - int result = Integer.parseInt(props.getProperty(CHUNK_SIZE_KEY, DEFAULT_CHUNK_SIZE + ""));
  + int result;
  + try {
  +     result = Integer.parseInt(props.getProperty(CHUNK_SIZE_KEY, DEFAULT_CHUNK_SIZE + ""));
  + } catch (final NumberFormatException ignore) {
  +     log.warn("'chunk-size' is not a valid number, use default value {}", DEFAULT_CHUNK_SIZE);
  +     return DEFAULT_CHUNK_SIZE;
  + }
  if (result <= 0) {
  ...
}
```

To make it easier for you to read, we have annotated the syntactic correctness of the generated DSLs:

Automatically generated DSL_A (Syntax Correct):

```dsl
functionDeclaration functiondeclaration_1 where
  and(
    functiondeclaration_1.body contain functionCall functioncall_1 where
    functioncall_1.name match "(?i)(parseInt|parseLong|parseDouble|parseFloat)",
    not(
      functiondeclaration_1.body contain exceptionBlock exceptionblock_1
    )
);
```

Automatically generated DSL_B (Syntax Error):

```dsl
```

```
functionDeclaration fd where
  and(
    fd contain functionCall fc where
    and(
      fc.name == "parseInt",
      fc.base.name == "Integer"
    ),
    not(
      fd contain tryStatement ts where
        ts contain catchClause cc where
          cc.exceptionType.name == "NumberFormatException"
    )
  );
```

Each DSL should be evaluated along two dimensions:
1. Readability: Is the generated DSL clear and easy to understand? Can you use it directly or with minimal modifications?
2. Generalizability: Based on your expertise, can the generated DSL be used to detect similar defects in different code contexts? [Matrix Scale Question]

Question\Options	Very Poor	Poor	Fair	Good	Excellent	Average Score
Readability of DSL_A	0(0%)	0(0%)	3(20%)	8(53.33%)	4(26.67%)	4.07
Generalizability of DSL_A	0(0%)	3(20%)	2(13.33%)	5(33.33%)	5(33.33%)	3.8
Readability of DSL_B	1(6.67%)	1(6.67%)	5(33.33%)	7(46.67%)	1(6.67%)	3.4
Generalizability of DSL_B	1(6.67%)	2(13.33%)	4(26.67%)	8(53.33%)	0(0%)	3.27
Subtotal	2(3.33%)	6(10%)	14(23.33%)	28(46.67%)	10(16.67%)	3.63

Q15. What do you think about the "intelligence" level of the automatically generated DSL_A in the above example? [Single Choice Question]

Options	Subtotal	Proportion
Meets my needs perfectly	1	6.67%
Produces readable output, but requires modifications	14	93.33%
Mostly redundant and ineffective	0	0%
Completely unreliable, still requires manual authoring	0	0%
Valid response count for this question	15	

Q16. What do you think about the "intelligence" level of the automatically generated DSL_B in the above example? [Single Choice Question]

Options	Subtotal	Proportion
Meets my needs perfectly	2	13.33%
Produces readable output, but requires modifications	9	60%
Mostly redundant and ineffective	3	20%
Completely unreliable, still requires manual authoring	1	6.67%
Valid response count for this question	15	

Q17. Based on your experience, please rank your preference for the following scenarios of applying automatically generated DSLs (1 = highest priority, 3 = lowest priority)
[Ranking Question]

Options	Overall Score	**1st Priority**	**2nd Priority**	**3rd Priority**	Subtotal
Manually refine the auto-generated valid DSLs to ensure they align with complex business logic.	2.8	12(80%)	3(20%)	0(0%)	15
DSLs and content are fully auto-generated (no manual intervention required).	1.6	3(25%)	6(50%)	3(25%)	12
Fully manually designed to avoid tool limitations.	0.93	0(0%)	3(27.27%)	8(72.73%)	11

Q18. What do you think are the advantages of automatically generating DSLs?[Multiple Selection Question]

Options	Subtotal	Proportion
Improve development efficiency	15	100%
Lower the barrier of domain knowledge	12	80%
Accelerate prototype design	10	66.67%
Other	2	13.33%
Valid response count for this question	15	

Q19. In what scenarios would you be willing to use automatically generated DSLs?[Multiple Selection Question]

Options	Subtotal	Proportion
Real-world production projects	12	80%
Rapid prototype development	14	93.33%
For learning or reference only	5	33.33%
Handling security/vulnerability warnings	2	13.33%
Currently unwilling to use	0	0%
Valid response count for this question	15	

Q20. Which aspect of the currently auto-generated DSLs do you think should be prioritized for improvement?[Multiple Selection Question]

Options	Subtotal	Proportion
Readability and ease of use	10	66.67%
Defect description capability	6	40%
Generalizability	11	73.33%
Query performance	7	46.67%
Other	1	6.67%
Valid response count for this question	15	